

# Analisis Arsitektur Aplikasi: Story Studio

## 1. Ringkasan Eksekutif

Story Studio adalah aplikasi berbasis Python (dengan antarmuka Streamlit) yang dirancang untuk membantu penulis dalam proses kreatif pembuatan cerita. Aplikasi ini mengelola "bible" cerita (karakter, latar, plot) dan hubungan antar karakter, serta menggunakan model bahasa besar (LLM) seperti Google Gemini atau model lokal via Ollama untuk menghasilkan konten naratif berdasarkan konteks yang tersimpan.

## 2. Arsitektur Sistem

Aplikasi ini menggunakan pendekatan modular dengan pemisahan tanggung jawab yang jelas:

- **UI Layer** (`app.py`): Menggunakan Streamlit untuk antarmuka pengguna, menangani interaksi input, dan menampilkan output.
- **Service Layer** (`story_service.py`): Mengatur alur logika bisnis utama (orchestrator), menghubungkan UI dengan provider AI dan manajemen data.
- **Provider Pattern** (`provider_factory.py`, `ai_provider.py`, `gemini_provider.py`, `ollama_provider.py`): Menggunakan **Factory Pattern** untuk mengabstraksi penyedia model AI. Hal ini memungkinkan aplikasi untuk beralih antara Gemini (cloud) dan Ollama (lokal) dengan mudah tanpa mengubah logika bisnis.
- **Data Layer** (`database.py`, `bible_manager.py`, `relationship_manager.py`): Mengelola penyimpanan dan pengambilan data cerita, karakter, dan hubungan menggunakan basis data SQLite.
- **Utility Layer** (`prompt_builder.py`, `summary_builder.py`, `system_prompts.py`): Memisahkan logika pembuatan prompt AI agar kode utama tetap bersih dan terfokus.

## 3. Diagram Alur Data (Data Flow)

1. **Entry Point** (`app.py`): Pengguna memilih proyek dan berinteraksi dengan UI.
2. **Request Initiation**: UI memanggil `StoryService` melalui `app.py`.
3. **Context Loading**: `StoryService` mengambil data dari `bible_manager` dan `relationship_manager` (database).
4. **Prompt Construction**: `prompt_builder` menggabungkan data cerita dengan *system prompts* untuk membentuk instruksi AI.
5. **Provider Invocation**: `provider_factory` menginisialisasi provider AI yang dipilih (Gemini/Ollama) dengan API Key atau konfigurasi lokal yang relevan.
6. **AI Inference**: Provider mengirimkan prompt ke LLM dan menerima respon.
7. **Response Delivery**: Respon diteruskan kembali melalui `StoryService` ke `app.py` untuk ditampilkan kepada pengguna.

## 4. Rincian File & Modul

| File                          | Deskripsi                                   |
|-------------------------------|---------------------------------------------|
| <code>app.py</code>           | Entry point aplikasi menggunakan Streamlit. |
| <code>story_service.py</code> | Orkesrator pusat, logika bisnis utama.      |

| File                                 | Deskripsi                                         |
|--------------------------------------|---------------------------------------------------|
| <code>provider_factory.py</code>     | Implementasi Factory Pattern untuk penyedia AI.   |
| <code>ai_provider.py</code>          | Kelas abstrak (interface) untuk provider AI.      |
| <code>gemini_provider.py</code>      | Implementasi provider untuk Google Gemini.        |
| <code>ollama_provider.py</code>      | Implementasi provider untuk model lokal (Ollama). |
| <code>database.py</code>             | Manajemen koneksi dan inisialisasi SQLite.        |
| <code>bible_manager.py</code>        | CRUD untuk data "bible" (karakter, latar, plot).  |
| <code>relationship_manager.py</code> | CRUD untuk hubungan antar karakter.               |
| <code>prompt_builder.py</code>       | Konstruksi prompt untuk LLM.                      |
| <code>system_prompts.py</code>       | Definisi prompt sistem (persona, aturan main).    |

## 5. Panduan Pertanyaan Interview

### 1. Tanya: "Mengapa Anda menggunakan Factory Pattern untuk penyedia AI?"

- *Jawaban:* Untuk memisahkan logika inisialisasi AI dari logika bisnis. Ini memungkinkan aplikasi mendukung berbagai model (Cloud/Lokal) secara *pluggable* tanpa merusak kode utama, serta mempermudah pengujian dengan *mocking* provider.

### 2. Tanya: "Bagaimana aplikasi menangani manajemen konteks cerita?"

- *Jawaban:* Konteks dikelola dengan cara mengambil data dari SQLite melalui `bible_manager` dan `relationship_manager`, kemudian data tersebut diformat menjadi struktur tekstual oleh `prompt_builder` sebelum dikirim ke AI sebagai bagian dari prompt (RAG-like approach).

### 3. Tanya: "Apa yang Anda lakukan jika API Key valid tetapi request gagal?"

- *Jawaban:* Implementasikan mekanisme *retry* dengan *exponential backoff* (sudah ada di `gemini_provider.py` untuk error 503) dan *logging* yang komprehensif untuk mendiagnosa apakah masalahnya adalah limitasi kuota, model yang sibuk, atau format data yang tidak didukung.

### 4. Tanya: "Bagaimana cara Anda menukar antara Gemini dan model lokal Ollama?"

- *Jawaban:* Cukup ubah variabel lingkungan `AI_PROVIDER` di file `.env`. `provider_factory` akan mendeteksi perubahan ini dan menginisialisasi kelas provider yang sesuai secara dinamis.

### 5. Tanya: "Mengapa memisahkan `prompt_builder` dari `story_service`?"

- *Jawaban:* *Separation of Concerns*. Pembuatan prompt bisa menjadi sangat kompleks seiring berkembangnya fitur. Dengan memisahkannya, kita bisa melakukan unit testing pada *prompting logic* tanpa harus menjalankan service atau melakukan panggilan ke AI secara nyata.